

SEMANTIC SEARCH INTELLIGENCE

Technical Project Brief

Semantic code search + grounded code explanation

1. Project Overview

Semantic Search Intelligence (SSI) is a developer-oriented system for finding and understanding code through natural-language search. Instead of requiring a user to know file names, classes, functions, or exact keywords, the system indexes source code into a vector database and retrieves code blocks that are semantically related to a query. Each result is presented with a confidence score to help the user judge relevance.

The system also includes an Explain mode. The retrieved code blocks are provided as context to an LLM, which answers a specific question about the code. The design supports both cloud-based LLMs and locally hosted models through Ollama, allowing the same workflow to be used with different model providers.

2. Problem Statement

Understanding an unfamiliar codebase is often slower than writing new code. Developers may know what they are looking for such as login logic, authentication, payment handling, or database access but not where that behavior lives. Traditional text search works best when the exact wording or symbol is already known.

SSI addresses this gap by combining semantic retrieval with LLM-based explanation. The vector search layer helps locate relevant code, while the explanation layer helps turn that retrieved code into a human-readable understanding of how the feature works.

3. Objectives

- Make code searchable using natural-language questions rather than exact keywords.
- Retrieve focused code blocks instead of forcing users to scan an entire repository.
- Provide a confidence score with search results to make relevance easier to judge.
- Use retrieved code as grounded context for explaining code behavior.
- Support both cloud LLMs and local LLMs through a model-agnostic interface.
- Improve onboarding, code understanding, documentation, and day-to-day code navigation.

4. High-Level Architecture

The system can be viewed as two connected pipelines: an indexing/retrieval pipeline and an explanation pipeline.

Layer	Responsibility	Main Technologies
Frontend	Search UI, result display, Explain mode, confidence presentation	TypeScript, React, Tailwind CSS, Zod
Backend/API	Request handling, orchestration, validation, search/explain flows	Python, FastAPI, Pydantic
Code processing	Parsing and structuring source code into searchable units	Tree-sitter
Data/search	Store and retrieve vector representations of code	Qdrant
Application data	Relational application data and	SQLAlchemy

	metadata	
LLM layer	Generate explanations from retrieved code context	Cloud LLMs or Ollama

The exact deployment topology can vary. The important design boundary is that retrieval happens first, and the LLM receives the retrieved code as context rather than being asked to reason about an unknown repository.

5. Search and Retrieval Flow

- 1. Index:** Source code is scanned and processed into searchable code units.
- 2. Parse:** Tree-sitter is used to understand source structure so that code can be handled as meaningful blocks rather than arbitrary text chunks.
- 3. Vectorize:** Code units are represented in a form suitable for semantic retrieval and stored in Qdrant along with relevant metadata.
- 4. Query:** A user asks a natural-language question, such as “Where is the login logic?”
- 5. Retrieve:** The system performs semantic search and returns the most relevant code blocks.
- 6. Rank / present:** Results are shown with a confidence score so users can quickly assess relevance.

6. Explain Mode

Explain mode sits on top of retrieval rather than replacing it. A user can ask a question such as “How is the login working?” after searching. The system takes the retrieved code blocks and supplies them to an LLM as context. The model then produces an explanation focused on the available code.

This approach is intended to keep explanations tied to the repository context. It also fits both hosted and local models: a cloud model can be used when convenience or model quality is preferred, while Ollama can be used when local execution or tighter control over code privacy is important.

7. Technology Stack

Technology	Role	Why it fits the project
Python	Backend and core processing	Strong ecosystem for APIs, data processing, and AI/ML integration.
FastAPI	Backend API	Lightweight API framework with good support for typed Python services.
Pydantic	Backend validation / schemas	Keeps request and response data structured and validated.
Tree-sitter	Source-code parsing	Provides syntax-aware parsing across supported programming languages.
Qdrant	Vector database	Designed for semantic/vector retrieval with metadata.
SQLAlchemy	Relational data access	Provides the database layer for application metadata and related data.
TypeScript	Frontend language	Adds type safety to the frontend codebase.
React	Frontend UI	Supports the interactive search and explanation experience.
Tailwind CSS	Frontend styling	Fast way to build and maintain the

		interface styling.
Zod	Frontend validation	Provides runtime validation for structured client-side data.
Ollama + cloud LLMs	Explanation layer	Allows the system to use either local or hosted models.

8. Grounded Answering Strategy

A core design principle of SSI is that the explanation layer should be grounded in retrieved code. The system prompt for the LLM is therefore intentionally strict: retrieved code is treated as the source of truth; missing information should be acknowledged; observed behavior should be separated from inference; and relevant files/symbols should be cited when possible.

This reduces the risk of an LLM confidently describing code that was not actually retrieved. Retrieval quality therefore remains a critical part of the overall system quality.

9. Primary Use Cases

- **Code search:** Find implementation details using questions written in natural language.
- **Code understanding:** Understand how a feature or flow works without manually tracing a large codebase.
- **Developer onboarding:** Help a new developer locate important parts of an unfamiliar repository.
- **Documentation support:** Generate explanations from existing implementation context.
- **Student / learner support:** Make existing projects easier to navigate and understand.
- **Team knowledge sharing:** Reduce time spent asking where a particular piece of logic lives.

10. Example User Flow

User query: “Where is the login logic?”

Expected interaction: the system returns the most relevant code blocks, identifies their source locations, and displays a confidence score for each result.

Explain mode query: “How is the login working?”

Expected interaction: the selected/retrieved code is passed to the LLM, which explains the login flow based on that context. Where the retrieved code does not provide enough evidence, the explanation should state that limitation instead of filling the gap with assumptions.

11. Scope and Current Direction

The current project direction is to build a useful semantic code search and explanation product rather than a generic chatbot. Retrieval is the foundation, and the LLM is an interpretation layer over the retrieved code.

A future production version can extend this foundation with stronger ranking, better repository indexing, multi-language support, evaluation datasets, usage analytics, model routing, authentication, team workspaces, and subscription-based access. These are product directions rather than claims about features already implemented.

12. Expected Impact

The intended impact is a reduction in the time required to locate and understand existing code. For developers and teams, this can improve onboarding and day-to-day code navigation. For students and learners, it provides a more direct way to explore real-world projects. Longer term, the same retrieval and explanation architecture can support a production service with hosted model infrastructure and subscription-based access.

13. Technical Notes and Constraints

- Semantic retrieval quality depends heavily on code parsing, chunking, embeddings, metadata, and ranking.
- Confidence scores should be treated as relevance signals rather than absolute correctness guarantees.
- LLM explanations are bounded by the code retrieved for the query; missing context can lead to incomplete explanations.
- Running local models through Ollama can provide a different privacy and cost profile from hosted LLM APIs.
- Production deployment will need additional controls around authentication, repository isolation, secrets, logging, and code privacy.